

Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Assignment No. 9 — Amortised Analysis

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Q1. Aggregate Analysis — Dynamic Array Expansion

Problem Statement:

A dynamic array doubles its capacity whenever it becomes full on insertion. Implement the array, track the actual cost of each insertion (1 for the insert itself, plus the cost of copying all existing elements if a resize happens), and use aggregate analysis to find the amortized cost per insertion over a sequence of n insertions.

Approach:

approach_1: maintain a capacity that starts at 1 and doubles whenever the array is full just before an insertion. the cost of an insertion that triggers a resize is (old capacity, for copying every existing element) + 1 (for the new element); an insertion that does not trigger a resize costs 1. keep a running total cost and, after every insertion, divide the running total by the number of insertions so far to report the amortized cost at that point.

aggregate analysis argument: across n insertions the array resizes at most $\log_2(n)$ times, and the total cost of all resizes is bounded by $1+2+4+\dots+n < 2n$ (a geometric series), so the total cost of n insertions is at most n (individual inserts) + $2n$ (copying) = $O(n)$. dividing by n gives an amortized cost of $O(1)$ per insertion even though some individual insertions cost $O(n)$.

Time and Space Complexity:

time complexity: $O(n)$

space complexity: $O(1)$

Code:

```
#include<iostream>
#include<vector>
using namespace std;
int main(){
    int n=16;
    int capacity=1;
    int size=0;
    long long totalCost=0;
    cout<<"operation\tactual cost\ttotal cost\tamortized cost"<<endl;
    for(int i=1;i<=n;i++){
        long long cost=1;
        if(size==capacity){
            cost+=capacity;
```

```

        capacity*=2;
    }
    size++;
    totalCost+=cost;
    double amortized=(double)totalCost/i;
    cout<<"insert "<<i<<"\t"<<cost<<"\t"<<totalCost<<"\t"<<amortized<<endl;
}
return 0;
}

```

Output:

```

*Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_9 at 11:19:32 PM
> mkdir -p build && g++ 1_dynamic_array_aggregate.cpp -o build/1_dynamic_array_aggregate && ./build/1_dynamic_array_aggregate
test case 5: n = 32
operation    actual cost    total cost    amortized cost
insert 1     1              1              1
insert 2     2              3              1.5
insert 3     3              6              2
insert 4     1              7              1.75
insert 5     5              12             2.4
insert 6     1              13            2.16667
insert 7     1              14             2
insert 8     1              15            1.875
insert 9     9              24            2.66667
insert 10    1              25            2.5
insert 11    1              26            2.36364
insert 12    1              27            2.25
insert 13    1              28            2.15385
insert 14    1              29            2.07143
insert 15    1              30             2
insert 16    1              31            1.9375
insert 17    17             48            2.82353
insert 18    1              49            2.72222
insert 19    1              50            2.63158
insert 20    1              51            2.55
insert 21    1              52            2.47619
insert 22    1              53            2.40909
insert 23    1              54            2.34783
insert 24    1              55            2.29167
insert 25    1              56            2.24
insert 26    1              57            2.19231
insert 27    1              58            2.14815
insert 28    1              59            2.10714
insert 29    1              60            2.06897
insert 30    1              61            2.03333
insert 31    1              62             2
insert 32    1              63            1.96875
final amortized cost per insertion: 1.96875

```

Q2. Accounting Method — Stack with Multipop Operation

Problem Statement:

Implement a stack-based browser back-button system supporting push(page), pop(), and multipop(k) (pop up to k items in one call). Using the accounting method, assign push an amortized cost of 2 units (1 for the actual push, 1 saved as credit) and pop (including each pop inside a multipop) an amortized cost of 0 units, paid for from the saved credit. Show that the total amortized cost of any sequence of n operations is $O(n)$.

Approach:

approach_1: every push places one credit unit on the item it pushes, so the credit bank always holds exactly as many units as there are items on the stack. a pop (standalone or as part of multipop(k)) never does new work beyond removing one item, so it is charged 0 amortized units and instead spends the 1 credit unit that was stored on that item when it was pushed.

the credit bank can never go negative because the number of items ever popped cannot exceed the number of items ever pushed, and each popped item consumes exactly the credit that its own push deposited.

this is simulated directly: each push records actual cost 1 and amortized charge 2 (bank += 1); each pop or each unit of a multipop records actual cost 1 and amortized charge 0 (bank -= 1 per item removed).

Time and Space Complexity:

time complexity: push is $O(1)$ actual cost; multipop(k) has an actual cost of $O(\min(k, \text{stack size}))$ but an amortized cost of $O(1)$ total (0 per popped item) because it is paid from pre-paid credit; over any sequence of n operations the total amortized cost is $O(n)$.

space complexity: $O(n)$ for the stack and the implicit credit bank counter.

Code:

```
#include<iostream>
#include<vector>
#include<string>
using namespace std;
class BrowserStack{
    vector<string>data;
public:
    long long actualTotal=0;
    long long amortizedTotal=0;
    long long creditBank=0;
    void push(string page){
        data.push_back(page);
        long long actual=1;
        long long amortized=2;
        creditBank+=(amortized-actual);
        actualTotal+=actual;
        amortizedTotal+=amortized;
        cout<<"push("<<page<<")\t\tactual cost "<<actual<<"\tamortized charge "<<amortized<<"\tcredit bank
after "<<creditBank<<endl;
    }
    void pop(){
        if(data.empty())
            return;
        data.pop_back();
        long long actual=1;
        long long amortized=0;
        creditBank-=1;
        actualTotal+=actual;
        amortizedTotal+=amortized;
        cout<<"pop()\t\tactual cost "<<actual<<"\tamortized charge "<<amortized<<"\tcredit bank after
"<<creditBank<<endl;
    }
    void multipop(int k){
        int popped=0;
        while(popped<k&&!data.empty()){
            data.pop_back();
            popped++;
        }
        long long actual=popped;
        long long amortized=0;
        creditBank-=popped;
        actualTotal+=actual;
        amortizedTotal+=amortized;
        cout<<"multipop("<<k<<")\t\tactual cost "<<actual<<"\tamortized charge "<<amortized<<"\tcredit
bank after "<<creditBank<<endl;
    }
};
int main(){
    BrowserStack s;
```

```

s.push("a");
s.push("b");
s.push("c");
s.push("d");
s.push("e");
s.multipop(3);
s.push("f");
s.pop();
s.multipop(10);
cout<<endl;
cout<<"total actual cost: "<<s.actualTotal<<endl;
cout<<"total amortized cost: "<<s.amortizedTotal<<endl;
cout<<"final credit bank: "<<s.creditBank<<endl;
return 0;
}

```

Output:

```

Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_9 at 11:21:33 PM
➔ mkdir -p build && g++ 2_stack_multipop_accounting.cpp -o build/2_stack_multipop_accounting && ./build/2_stack_multipop_accounting
pop()          actual cost 1   amortized charge 0   credit bank after 1
push(p3)       actual cost 1   amortized charge 2   credit bank after 2
push(p4)       actual cost 1   amortized charge 2   credit bank after 3
push(p5)       actual cost 1   amortized charge 2   credit bank after 4
multipop(5)    actual cost 4   amortized charge 0   credit bank after 0
total actual cost: 10
total amortized cost: 10
final credit bank: 0

test case 4:
multipop(5)    actual cost 0   amortized charge 0   credit bank after 0
pop()          actual cost 0   amortized charge 0   credit bank after 0
push(q1)       actual cost 1   amortized charge 2   credit bank after 1
pop()          actual cost 1   amortized charge 0   credit bank after 0
total actual cost: 2
total amortized cost: 2
final credit bank: 0

test case 5:
push(1)        actual cost 1   amortized charge 2   credit bank after 1
push(2)        actual cost 1   amortized charge 2   credit bank after 2
push(3)        actual cost 1   amortized charge 2   credit bank after 3
push(4)        actual cost 1   amortized charge 2   credit bank after 4
push(5)        actual cost 1   amortized charge 2   credit bank after 5
push(6)        actual cost 1   amortized charge 2   credit bank after 6
push(7)        actual cost 1   amortized charge 2   credit bank after 7
push(8)        actual cost 1   amortized charge 2   credit bank after 8
push(9)        actual cost 1   amortized charge 2   credit bank after 9
push(10)       actual cost 1   amortized charge 2   credit bank after 10
multipop(4)    actual cost 4   amortized charge 0   credit bank after 6
multipop(4)    actual cost 4   amortized charge 0   credit bank after 2
multipop(4)    actual cost 2   amortized charge 0   credit bank after 0
total actual cost: 20
total amortized cost: 20
final credit bank: 0

```

Q3. Dynamic Tables — Table Expansion and Contraction

Problem Statement:

Implement a dynamic table that doubles its size when an INSERT is attempted on a full table and halves its size when a DELETE drops the table below a chosen load-factor threshold. Compare Policy A (contract as soon as the table is half-full, load factor 1/2) against Policy B (contract only once the table is one-quarter full, load factor 1/4) on an adversarial sequence that alternates one DELETE and one INSERT right at Policy A's resize boundary, and report the total and average cost per operation for each policy.

Approach:

approach_1: represent the table by a (size, count) pair. insertItem() costs 1 if count < size; if the table is full it first costs size (to copy every existing element into a table of double the size) before costing the extra 1 for the new element, and only then increments count. deleteItem() always costs 1 to remove the element, then checks the load factor count/size against the policy's threshold (1/2 for policy A, 1/4 for policy B); if the load factor has dropped to or below the threshold, it additionally costs size/2 to copy every remaining element into a table of half the size.

approach_2 (adversarial sequence): first delete elements from a full table of size 16 down to the load factor that sits exactly at policy A's contraction boundary, then alternate insert and delete at that same boundary for the rest of the 30 operations. under policy A this makes every single insert or delete trigger a full $O(\text{size})$ resize — called thrashing — because the load factor keeps crossing the $1/2$ threshold back and forth. under policy B the extra gap between the doubling threshold (full) and the halving threshold ($1/4$ full) means the same alternating sequence never crosses a resize boundary, so every operation stays $O(1)$.

Time and Space Complexity:

time complexity: $O(1)$ amortized per operation for both policies over a general (non-adversarial) sequence, using the same doubling/halving aggregate argument as Q1; however on the adversarial sequence described above, policy A degrades to $O(\text{size})$ per operation (thrashing) while policy B remains $O(1)$ amortized because its contraction threshold leaves a safety margin that an alternating insert/delete sequence cannot repeatedly cross.

space complexity: $O(\text{size})$ for the table at any point.

Code:

```
#include<iostream>
#include<vector>
#include<string>
using namespace std;
enum Policy{POLICY_A,POLICY_B};
class DynamicTable{
    int size;
    int count;
    Policy policy;
public:
    long long totalCost=0;
    int opCount=0;
    DynamicTable(int startSize,Policy p){
        size=startSize;
        count=startSize;
        policy=p;
    }
    long long insertItem(){
        long long cost=1;
        if(count==size){
            cost+=size;
            size*=2;
        }
        count++;
        totalCost+=cost;
        opCount++;
        return cost;
    }
    long long deleteItem(){
        long long cost=1;
        if(count>0)
            count--;
        double loadFactor=(size>0)?(double)count/size:0;
        double threshold=(policy==POLICY_A)?0.5:0.25;
        if(size>1&&loadFactor<=threshold){
            cost+=size/2;
            size/=2;
        }
        totalCost+=cost;
        opCount++;
        return cost;
    }
}
```

```

    int getSize(){return size;}
    int getCount(){return count;}
};
vector<string>buildAdversarialSequence(int totalOps){
    vector<string>ops;
    for(int i=0;i<8;i++){
        ops.push_back("delete");
    }
    while((int)ops.size()<totalOps){
        ops.push_back("insert");
        if((int)ops.size()<totalOps)
            ops.push_back("delete");
    }
    return ops;
}
void runPolicy(string label,Policy p,vector<string>&ops){
    DynamicTable table(16,p);
    cout<<"policy "<<label<<endl;
    cout<<"operation\tsize\tcount\tcost"<<endl;
    for(string op:ops){
        long long cost;
        if(op=="insert")
            cost=table.insertItem();
        else
            cost=table.deleteItem();
        cout<<op<<"\t\t"<<table.getSize()<<"\t"<<table.getCount()<<"\t"<<cost<<endl;
    }
    double avg=(double)table.totalCost/table.opCount;
    cout<<"total cost ("<<table.opCount<<" ops): "<<table.totalCost<<endl;
    cout<<"average cost per operation: "<<avg<<endl;
    cout<<endl;
}
int main(){
    int totalOps=30;
    vector<string>ops=buildAdversarialSequence(totalOps);
    runPolicy("A - naive (contract at 1/2 full)",POLICY_A,ops);
    runPolicy("B - safe (contract at 1/4 full)",POLICY_B,ops);
    return 0;
}

```

Output:

```
Linux@atharvk % main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_9 at 11:23:55 PM
➔ mkdir -p build && g++ 3_dynamic_table_policies.cpp -o build/3_dynamic_table_policies && ./build/3_dynamic_table_policies
test case 1: start size = 16, total operations = 30
policy A - naive (contract at 1/2 full)
total cost (30 ops): 214
average cost per operation: 7.13333
policy B - safe (contract at 1/4 full)
total cost (30 ops): 30
average cost per operation: 1

test case 2: start size = 8, total operations = 20
policy A - naive (contract at 1/2 full)
total cost (20 ops): 88
average cost per operation: 4.4
policy B - safe (contract at 1/4 full)
total cost (20 ops): 20
average cost per operation: 1

test case 3: start size = 32, total operations = 40
policy A - naive (contract at 1/2 full)
total cost (40 ops): 440
average cost per operation: 11
policy B - safe (contract at 1/4 full)
total cost (40 ops): 40
average cost per operation: 1

test case 4: start size = 16, total operations = 10
policy A - naive (contract at 1/2 full)
total cost (10 ops): 34
average cost per operation: 3.4
policy B - safe (contract at 1/4 full)
total cost (10 ops): 10
average cost per operation: 1

test case 5: start size = 64, total operations = 50
policy A - naive (contract at 1/2 full)
total cost (50 ops): 658
average cost per operation: 13.16
policy B - safe (contract at 1/4 full)
total cost (50 ops): 50
average cost per operation: 1
```

Q4. Accounting Method — Queue Using Two Stacks

Problem Statement:

Build a FIFO queue out of two stacks, inStack and outStack, for a print spooler. enqueue(x) pushes x onto inStack. dequeue() moves every item from inStack to outStack (reversing their order) only when outStack is empty, then pops and returns the top of outStack. Using the accounting method, charge enqueue 2 amortized units (1 for the push, 1 saved as credit for that item's later move to outStack) and dequeue 1 amortized unit (the move of each item out of inStack is paid for by that item's own saved credit, not by the dequeue call that triggers the move).

Approach:

approach_1: every enqueue deposits 1 credit unit on the item it pushes (amortized charge 2, actual cost 1, so 1 unit goes to the bank). when a dequeue call finds outStack empty, it moves every item currently in inStack over to outStack one at a time; each such move is paid for by spending the 1 credit unit that item was given back when it was enqueued, so the transfer itself is amortized-free. the final pop off outStack (returning the answer) is the only part of dequeue charged a fresh amortized unit, fixed at 1 regardless of how many items were transferred underneath it.

because every item can be transferred from inStack to outStack at most once over its lifetime, and that one transfer is exactly what its enqueue credit was saved for, the credit bank never goes negative and the total amortized cost of any sequence of n enqueue/dequeue operations is $O(n)$, even though an individual dequeue that triggers a full transfer has an actual cost of $O(\text{size of inStack})$.

Time and Space Complexity:

time complexity: enqueue is $O(1)$ actual and amortized; dequeue is $O(1)$ amortized but $O(n)$ actual in the worst case when it triggers a full transfer; over any sequence of n operations the total amortized (and hence total actual) cost is $O(n)$.

space complexity: $O(n)$ for the two stacks combined.

Code:

```
#include<iostream>
#include<vector>
#include<string>
using namespace std;
class QueueTwoStacks{
    vector<int>inStack;
    vector<int>outStack;
public:
    long long actualTotal=0;
    long long amortizedTotal=0;
    long long creditBank=0;
    void enqueue(int x){
        inStack.push_back(x);
        long long actual=1;
        long long amortized=2;
        creditBank+=(amortized-actual);
        actualTotal+=actual;
        amortizedTotal+=amortized;
        cout<<"enqueue("<<x<<")\t\tactual cost "<<actual<<"\tamortized charge "<<amortized<<"\tcredit bank
after "<<creditBank<<endl;
    }
    int dequeue(){
        long long actual=0;
        if(outStack.empty()){
            while(!inStack.empty()){
                outStack.push_back(inStack.back());
                inStack.pop_back();
                actual++;
                creditBank-=1;
            }
        }
        int result=-1;
        if(!outStack.empty()){
            result=outStack.back();
            outStack.pop_back();
            actual++;
        }
        long long amortized=1;
        amortizedTotal+=amortized;
        cout<<"dequeue() -> "<<result<<"\tactual cost "<<actual<<"\tamortized charge
"<<amortized<<"\tcredit bank after "<<creditBank<<endl;
        return result;
    }
};

int main(){
    QueueTwoStacks q;
    for(int i=1;i<=5;i++)
        q.enqueue(i);
    q.dequeue();
    cout<<endl;
    cout<<"total actual cost: "<<q.actualTotal<<endl;
    cout<<"total amortized cost: "<<q.amortizedTotal<<endl;
    cout<<"final credit bank: "<<q.creditBank<<endl;
    return 0;
}
```

Output:

```
Linux@atharvk: ~$ main in ~/Documents/WCE Lab Material/TY Computer Algorithm Lab/245109074_CAL Assignment 9 at 11:25:15 PM
➤ mkdir -p build && g++ 4_queue_two_stacks_accounting.cpp -o build/4_queue_two_stacks_accounting && ./build/4_queue_two_stacks_accounting

test case 1:
enqueue(1)      actual cost 1  amortized charge 2  credit bank after 1
enqueue(2)      actual cost 1  amortized charge 2  credit bank after 2
enqueue(3)      actual cost 1  amortized charge 2  credit bank after 3
enqueue(4)      actual cost 1  amortized charge 2  credit bank after 4
enqueue(5)      actual cost 1  amortized charge 2  credit bank after 5
dequeue() -> 1   actual cost 6  amortized charge 1  credit bank after 0
total actual cost: 11
total amortized cost: 11
final credit bank: 0

test case 2:
enqueue(1)      actual cost 1  amortized charge 2  credit bank after 1
enqueue(2)      actual cost 1  amortized charge 2  credit bank after 2
enqueue(3)      actual cost 1  amortized charge 2  credit bank after 3
dequeue() -> 1   actual cost 4  amortized charge 1  credit bank after 0
dequeue() -> 2   actual cost 1  amortized charge 1  credit bank after 0
enqueue(4)      actual cost 1  amortized charge 2  credit bank after 1
dequeue() -> 3   actual cost 1  amortized charge 1  credit bank after 1
total actual cost: 10
total amortized cost: 11
final credit bank: 1

test case 3:
dequeue() -> queue is empty  actual cost 0  amortized charge 0  credit bank after 0
total actual cost: 0
total amortized cost: 0
final credit bank: 0
```

```
test case 3:
dequeue() -> queue is empty      actual cost 0   amortized charge 0   credit bank after 0
total actual cost: 0
total amortized cost: 0
final credit bank: 0
```

```
test case 4:
enqueue(1)      actual cost 1   amortized charge 2   credit bank after 1
enqueue(2)      actual cost 1   amortized charge 2   credit bank after 2
enqueue(3)      actual cost 1   amortized charge 2   credit bank after 3
enqueue(4)      actual cost 1   amortized charge 2   credit bank after 4
enqueue(5)      actual cost 1   amortized charge 2   credit bank after 5
enqueue(6)      actual cost 1   amortized charge 2   credit bank after 6
enqueue(7)      actual cost 1   amortized charge 2   credit bank after 7
enqueue(8)      actual cost 1   amortized charge 2   credit bank after 8
enqueue(9)      actual cost 1   amortized charge 2   credit bank after 9
enqueue(10)     actual cost 1   amortized charge 2   credit bank after 10
dequeue() -> 1   actual cost 11  amortized charge 1   credit bank after 0
dequeue() -> 2   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 3   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 4   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 5   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 6   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 7   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 8   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 9   actual cost 1   amortized charge 1   credit bank after 0
dequeue() -> 10  actual cost 1   amortized charge 1   credit bank after 0
total actual cost: 30
total amortized cost: 30
final credit bank: 0
```

```
test case 5:
enqueue(1)      actual cost 1   amortized charge 2   credit bank after 1
dequeue() -> 1   actual cost 2   amortized charge 1   credit bank after 0
enqueue(2)      actual cost 1   amortized charge 2   credit bank after 1
dequeue() -> 2   actual cost 2   amortized charge 1   credit bank after 0
enqueue(3)      actual cost 1   amortized charge 2   credit bank after 1
dequeue() -> 3   actual cost 2   amortized charge 1   credit bank after 0
enqueue(4)      actual cost 1   amortized charge 2   credit bank after 1
dequeue() -> 4   actual cost 2   amortized charge 1   credit bank after 0
total actual cost: 12
total amortized cost: 12
final credit bank: 0
```